Skyjo

Web-Services Projekt

Tobias Gramlich, Mona Ittlinger, Melina Klein

Agenda

- Überblick über das Gesamtsystem
- Frontend
- Matchmaking
- Benutzerverwaltung
- Skyjo-Logic (mit Ausblick)
- Color-Weather-API

Überblick über das Gesamtsystem

Grundidee

- Kartenspiel Skyjo implementieren
- Benutzerverwaltung mit Authentifizierungsfunktion
- Matchmaking für dynamische Suche von Gegnern
- Color-Weather-API, die ein angepasstes Farbschema bereitstellt

Services

Aus der Grundidee haben sich dann 4 Services entwickelt:

- Weather-Color-Api
- Benutzerverwaltung
- Matchmaking
- Skyjo-Logic

Arbeitsaufteilung?

Die Arbeitsaufteilung:

- Melina Klein: Weather-Color-API, Skyjo-Logic
- Mona Ittlinger: Matchmaking, Weboberfläche
- Tobias Gramlich: Benutzerverwaltung, Matchmaking, Skyjo-Logic

Frontend

Überblick

Frontend

Idee:

Die bestehenden Services darstellen

Technologien:

React	als Framework
Javascript	als Programmiersprache

Matchmaking

Überblick

Matchmaking

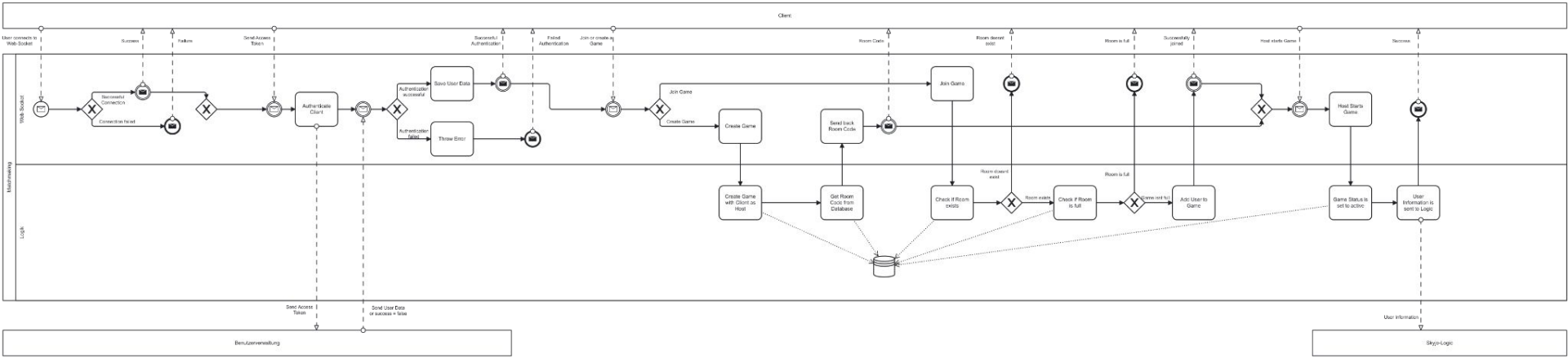
Idee:

Ein Service, in dem sich Spieler in privaten Spielräumen treffen können und gegeneinander spielen können.

Technologien:

MariaDB	als Datenbank
Javascript	für die Backend-Logik
WebSocket	zum Hosten von Echtzeit-Sitzungen

BPMN


Client
User connects to Web Socket
Successful Connection
Connection failed
Success
Failure
Send Access Token
Authenticate Client
Authentication successful
Authentication failed
Save User Data
Throw Error
Successful Authentication
Failed Authentication
Join or create a Game
Join Game
Create Game
Create Game
Send back Room Code
Room Code
Join Game
Room doesn't exist
Room is full
Successfully joined
Host starts Game
Success
Host Starts Game
Web Socket
Matchmaking
Logic
Create Game with Client as Host
Get Room Code from Database
Check if Room exists
Room doesn't exist
Room exists
Check if Room is full
Room is full
Game not full
Add User to Game
Game Status is set to active
User Information is sent to Logic
Send Access Token
Send User Data or success = false
User Information
Benutzerverwaltung
Skyjo-Logic

Web-Socket-Kommunikation

Funktionen, die über das Web-Socket erreichbar sind:

- user.authenticate: Für die Authentifizierung des Nutzers
- private.create: Für das Erstellen eines Raumes
- private.join: Für das Beitreten in einen Raum
- private.leave: Für das Verlassen eines Raumes
- private.start: Für das Starten des Spieles

Benutzerverwaltung

Überblick

Benutzerverwaltung

Idee:

Ein Service, in welchem sich Nutzer registrieren können und welcher diese auch authentifiziert.

Technologien:

MariaDB	als Datenbank
Javascript	für die Backend-Logik
Node.js	als Laufzeitumgebung
Express	für die REST-API

REST-API-Endpunkte

Grund: Einfacher Austausch von Daten als JSON-Objekt

Wichtigste Endpunkte:

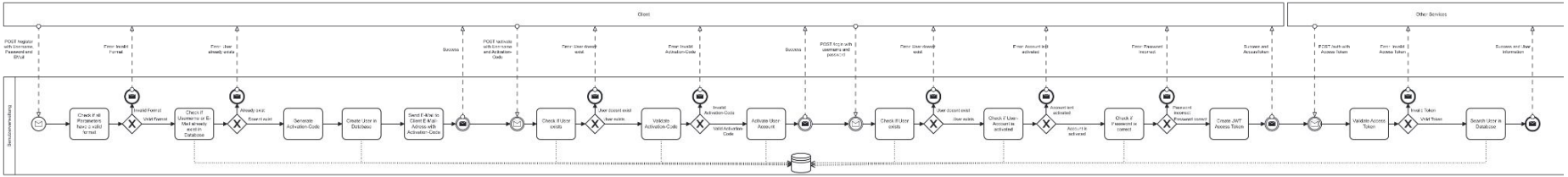
- POST /register: Registriert einen neuen Nutzer mit Username, Passwort und E-Mail
- POST /activate: Aktiviert den Nutzeraccount mit dem Aktivierungscode
- POST /login: Gibt dem Benutzer ein JWT Access Token, mit dem er auf Funktionen zugreifen kann
- POST /auth: Verifiziert das Access Token eines Nutzers und gibt an, um welchen Nutzer es sich handelt

Weitere Endpunkte

Weitere funktionierende Endpunkte:

- POST /send_email: Versendet die Aktivierungsmail erneut
- PUT /change_username: Ändert den Benutzernamen
- PUT /change_password: Ändert das Passwort
- DELETE /delete_account: Löscht den Nutzeraccount

Registrierungsablauf BPMN



skyjo-logic

Überblick

skyjo-logic

Idee:

Ein Service, der die Spiellogik von Skyjo implementiert.

Technologien:

MariaDB	als Datenbank
Java	für die Businesslogik
Quarkus	für die API
Jooq	als SQL Generator und Mapper

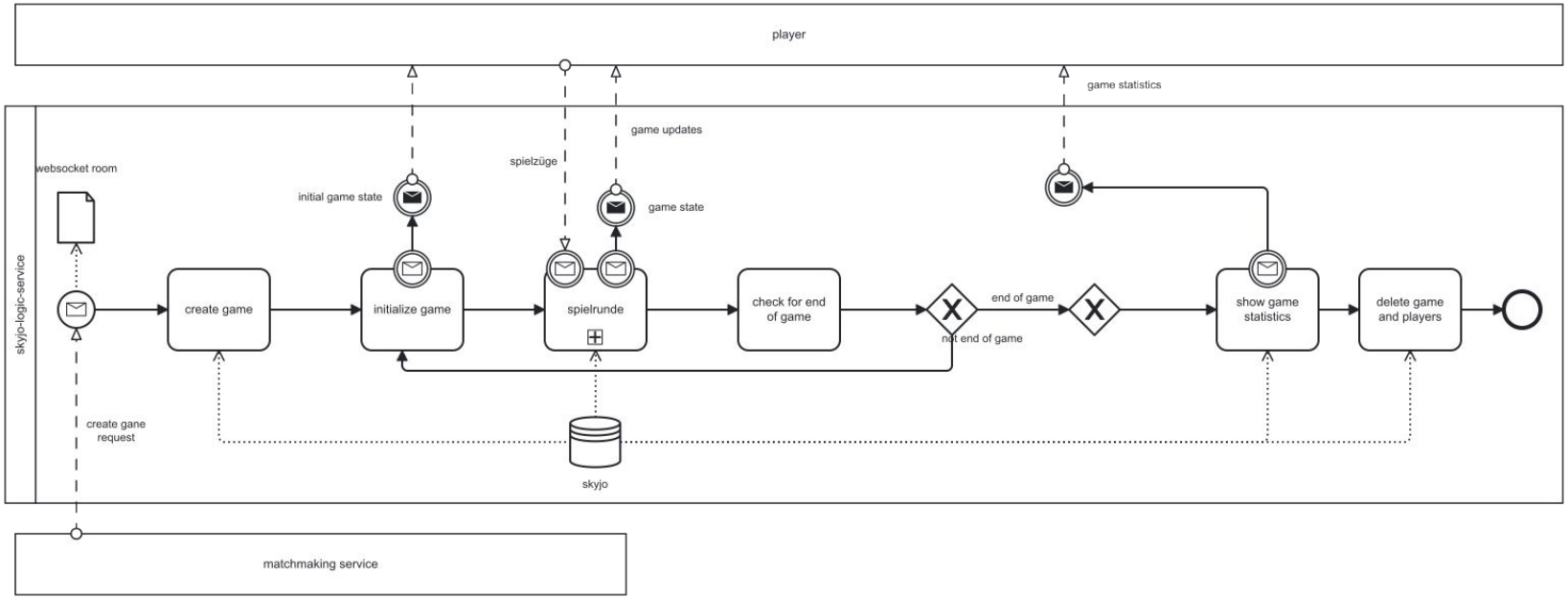
Websocket

Grund: einfache Möglichkeit, die UI zu aktualisieren und um anzuzeigen, wer wann an der Reihe ist.

Umsetzung: Raum wird von Matchmaking Service übergeben und die Logik nutzt diesen dann weiter.

Hierüber werden Realtime-Updates geschickt, die Veränderungen in der UI bedeuten.

Spielablauf BPMN



API-Endpunkte

- **POST /move**: sendet eine Move-Anfrage und bekommt zurück ob der Move richtig ist und durchgeführt wird oder nicht. Wenn er nicht gültig ist, wird eine Error Message mitgeliefert.
- **POST /setUpGame**: bekommt eine Liste an Spielern überreicht. Für diese wird ein Spiel erstellt und dieses zurückgegeben.
- **GET /getGame-{id}**: gibt den aktuellen Spielstand eines Spiels zurück und wird nach einem Verbindungsverlust benutzt.

Zwischenstand

- es gibt funktionierende API-Punkte mit den die Logik getestet werden kann
- die Datenbank existiert und läuft
- die Logik wurde implementiert

Ausblick

Was muss noch getan werden?

- andere Services anbinden um Nutzer zu authentifizieren
- WebSocket von Matchmaking bekommen
- WebSocket-Updates erstellen

weather-color-api

Überblick

Weather-Color-API

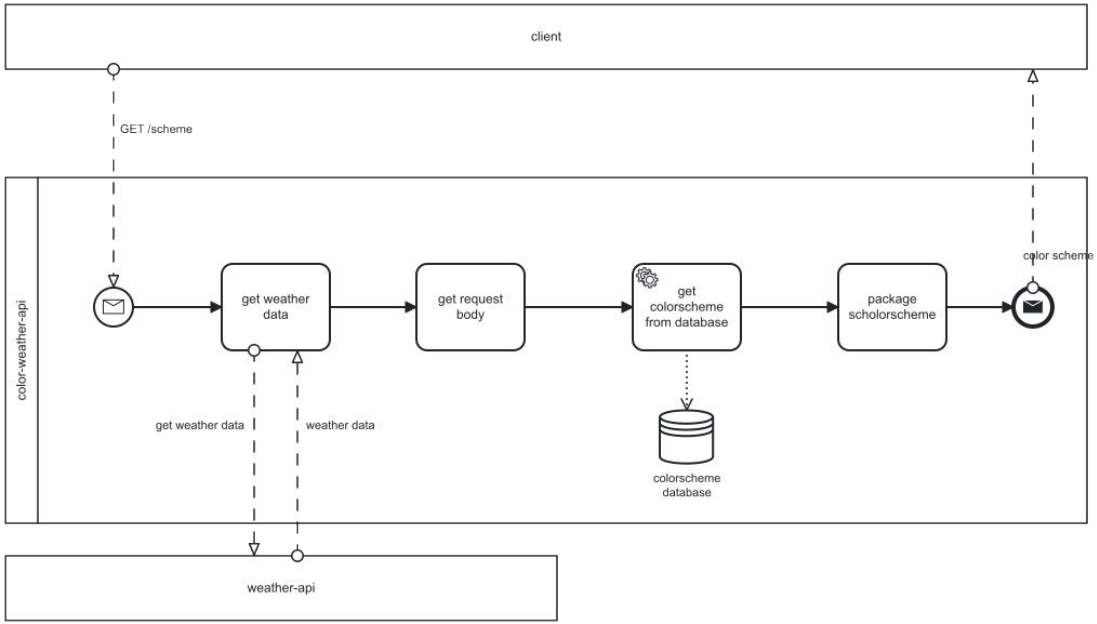
Idee:

Eine leichtgewichtige API, die basierend auf der aktuellen Uhrzeit und des Wetters ein passendes Farbschema zurückgibt.

verwendete Technologien:

MariaDB	als Datenbank
Python	für die Backend-Logik
FastAPI	für die Bereitstellung der API-Schnittstelle

Ablauf eines `GET /scheme` Requests



vorhandene Requests an die REST-API

OpenAPI Dokumentation (Backup)

GET `/scheme` Get Scheme

Return a scheme based on current weather + local time.

Parameters | Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header.	*No links*

Example Value | Schema


```
{
  "request": {
    "time_of_day": "Morning",
    "day_type": "Weekday",
    "weather": "Sunny"
  },
  "scheme": {
    "background": "string",
    "surface": "string",
    "primary_color": "string",
    "secondary_color": "string"
  }
}
```

OpenAPI Dokumentation (Backup)

POST `/scheme` Get Scheme For Request

Return a scheme for a given explicit request.

Parameters — Try it out — Reset

No parameters

Request body required — application/json

Example Value Schema

```
{
  "time_of_day": "Midday",
  "day_type": "Weekday",
  "weather": "Cloudy"
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/scheme' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "time_of_day": "Midday",
  "day_type": "Weekday",
  "weather": "Cloudy"
}'
```

Request URL

```
http://localhost:8000/scheme
```

Server response

Code	Details
200	Response body

```
{
  "background": "#f4f6f8",
  "surface": "white",
  "primary_color": "#90a4ae",
  "secondary_color": "#b0bec5"
}
```

Download

Response headers

```
content-length: 96
content-type: application/json
date: Tue,10 Mar 2026 13:32:24 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header.

Example Value Schema

```
{
  "background": "string",
  "surface": "string",
  "primary_color": "string",
  "secondary_color": "string"
}
```

422	Validation Error	No links

Media type: application/json

Example Value Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

Ausblick